

SPIR-V Cooperative Matrices:

SPV_KHR_cooperative_matrix and Cooperative Matrix 2 (NV) — Performance Limits and Backend Mapping

Executive summary

The SPIR-V cooperative matrix model (as standardized in `SPV_KHR_cooperative_matrix`) introduces an *abstract* matrix type whose storage and computation are distributed across invocations in a declared scope (commonly a subgroup). The core contract deliberately leaves supported shapes (MxNxK), element types, and some precision semantics to the client API (e.g., Vulkan) and the implementation, while enforcing “everyone participates” semantics (uniform operands within scope; all invocations active). ¹

In Vulkan, that abstraction is made practical (and constrained) by runtime queries describing which combinations of `(AType, BType, CType, ResultType, MSize, NSize, KSize, scope, saturatingAccumulation)` are supported (`VkCooperativeMatrixPropertiesKHR`), and by additional SPIR-V environment validity rules that require declared matrices and their scope to match those reported capabilities. ²

“Cooperative Matrix 2” in current public shipping form is *primarily* exposed via the vendor extension `VK_NV_cooperative_matrix2`, which depends on the SPIR-V extensions `SPV_NV_cooperative_matrix2` and `SPV_NV_tensor_addressing`, and a companion GLSL extension. This “v2” direction targets a key limitation of baseline cooperative matrices: opaque matrix objects are great for $D = A \cdot B + C$, but real ML kernels need extra conversions, reductions, and richer memory/addressing controls. ³

From a codegen perspective, peak performance on modern GPUs generally requires lowering cooperative-matrix operations into each vendor’s matrix ISA path: for NVIDIA GPUs, PTX provides tensor-core instructions such as `mma` plus related matrix load instructions such as `ldmatrix` (and later generations add “warp-group” MMA variants); for AMD GPUs, CDNA2 documents MFMA (“Matrix-Fused-Multiply-Add”) instructions, while RDNA3 exposes WMMA as a feature usable from HIP via compiler intrinsics. ⁴

Tooling reality: standardized `SPV_KHR_cooperative_matrix` has broad ecosystem visibility (validators, Vulkan env rules, etc.), while “Cooperative Matrix 2” (`SPV_NV_cooperative_matrix2`) is still catching up in open compiler pipelines—e.g., as of late 2025 an issue remains open asking whether `SPV_NV_cooperative_matrix2` / `SPV_NV_tensor_addressing` are in scope for `SPIRV-LLVM-Translator`. ⁵

Specifications and extension ecosystem

Baseline cooperative matrices are defined in SPIR-V by `SPV_KHR_cooperative_matrix`: it adds a cooperative matrix type (`OpTypeCooperativeMatrixKHR`), load/store (`OpCooperativeMatrixLoadKHR` / `OpCooperativeMatrixStoreKHR`), multiply-add (`OpCooperativeMatrixMulAddKHR`), and a “length” query (`OpCooperativeMatrixLengthKHR`). It also specifies that cooperative matrices behave like composites with an implementation-defined number of per-invocation components, and it enforces critical uniformity/participation rules: operands must be dynamically uniform within each scope instance; dynamic execution must ensure either all invocations in the scope instance are active or none are. ⁶

The baseline SPIR-V extension explicitly *does not* define what matrix shapes and type combinations are legal; that is left to the client API. Vulkan’s cooperative matrix extension (`VK_KHR_cooperative_matrix`) supplies:

- A definition-level description of cooperative matrix intent (“medium-sized matrices”, “primarily supported in compute shaders”, spread across invocations in a scope). ⁷

- A runtime query mechanism (`VkCooperativeMatrixPropertiesKHR`) that enumerates supported {MSize, NSize, KSize, AType, BType, CType, ResultType, scope} and whether saturating accumulation must/must-not be requested for a given combination. ⁸

- SPIR-V environment validity rules that tie `OpTypeCooperativeMatrixKHR` declarations to the reported properties, and (for flexible-dimension variants) require scope/workgroup invocation counts to match reported values. ⁹

“Cooperative Matrix 2” (as the user referenced it) currently maps most directly onto `VK_NV_cooperative_matrix2`. The Vulkan repage states that the “new functionality” is defined by the SPIR-V extensions `SPV_NV_tensor_addressing` and `SPV_NV_cooperative_matrix2`, and is usable via `GLSL_NV_cooperative_matrix2`; it also highlights that this extension supports enumerating matrix types/dimensions and “which specific features are supported.” ¹⁰

While a standalone HTML extension spec for `SPV_NV_cooperative_matrix2` itself was not discoverable in the Khronos SPIR-V Registry HTML outputs during this session, multiple primary sources still expose its surface area indirectly: the SPIR-V unified specification enumerates capabilities and opcodes referencing cooperative-matrix reductions and tensor-addressing operands (e.g., `OpCooperativeMatrixReduceNV`, `OpCooperativeMatrixLoadTensorNV` / `OpCooperativeMatrixStoreTensorNV`, and related operand enums). ¹¹

Separately, `SPV_NV_tensor_addressing` is published with a full extension spec, defining “tensor layout” and “tensor view” types as opaque representations to translate matrix/tensor coordinates into linear memory layout for cooperative matrix load/store paths. ¹²

For comparison, `SPV_INTEL_joint_matrix` (a preview extension specification from Intel ¹³) defines a *joint matrix* type and a bundle of joint-matrix load/store and matrix MAD-family operations. It includes explicit layout enumerants (including packed layouts described as suitable for VNNI), and it introduces a synchronization scope operand on its load/store operations (separate from the storage scope baked into the type). ¹⁴

On the compiler side, MLIR work discussions reference this spec directly when modeling SPIR-V joint-matrix types/ops. ¹⁵

Upstream LLVM SPIR-V backend documentation also lists `SPV_INTEL_joint_matrix` among supported

extensions and summarizes it as adding capabilities “on top of `SPV_KHR_cooperative_matrix`” (e.g., prefetch, coordinate queries, checked ops, and TF32/bfloat interpretations for multiply-add). ¹⁶

A note on the user-requested “LLVM/Intel PR #12497”: public web search in this session did not locate an unambiguous cooperative-matrix-related change request under that identifier. Multiple unrelated “#12497” items exist across unrelated projects, and no primary source tied that number to cooperative matrices here. (Accordingly, this report does not treat it as a verified citation anchor.) ¹⁷

Cooperative Matrix 2 killer features

What follows summarizes “killer features” as evidenced by the public Vulkan extension documentation for `VK_NV_cooperative_matrix2`, the published `SPV_NV_tensor_addressing` spec, and the SPIR-V unified opcode/capability tables that reference `SPV_NV_cooperative_matrix2`. ¹⁸

More than GEMM: conversions, per-element ops, and reductions.

Baseline cooperative matrices are designed around `OpCooperativeMatrixMulAddKHR` plus load/store and basic arithmetic/conversions on the matrix composite type. ¹⁹

However, extension proposals around cooperative matrices explicitly highlight that many ML kernels (convolutions, LLMs) need additional manipulation not directly supported by opaque cooperative matrix objects. ²⁰

Within SPIR-V’s unified opcode tables, `OpCooperativeMatrixReduceNV` exists and is associated with a “Cooperative Matrix Reduce Mode” mask that supports reductions over rows, columns, or 2x2 groupings—indicating an explicit reduction capability beyond baseline GEMM. ²¹

The unified capability list also reserves multiple NV capabilities—`CooperativeMatrixReductionsNV`, `CooperativeMatrixConversionsNV`, `CooperativeMatrixPerElementOperationsNV`, `CooperativeMatrixTensorAddressingNV`, and `CooperativeMatrixBlockLoadsNV`—as “also see `SPV_NV_cooperative_matrix2`,” signaling the intended feature grouping in v2. ²²

Richer memory layout and addressing via tensor layout/view.

`SPV_NV_tensor_addressing` defines “tensor layout” and “tensor view” opaque types and describes them as mapping between matrix coordinates and tensor memory layout, supporting multi-dimensional layouts (rank 1–5). This is explicitly positioned as an enabling technology “initially” used with `SPV_NV_cooperative_matrix2`. ¹²

The SPIR-V unified specification defines a “Tensor Addressing Operands” literal mask used by NV cooperative-matrix tensor load/store instructions, including a `TensorView` operand bit and a `DecodeFunc` bit (associated with “block loads”). ²³

The Vulkan SPIR-V environment spec further nails down arithmetic details for these tensor-coordinate-to-linear computations: if 64-bit indexing is enabled, calculations used by `OpCooperativeMatrixLoadTensorNV` and `OpCooperativeMatrixStoreTensorNV` are performed with 64-bit range for the result. ²⁴

Layouts, tiling, and runtime-enumerated legality remain central.

Even with v2 features, legality is still expected to be enumerated/queried rather than assumed. Vulkan’s cooperative-matrix model stresses that SPIR-V defines types/instructions but expects implementations to support different restricted sets of sizes and combinations. ²⁵

For baseline KHR cooperative matrices, the SPIR-V extension defines a `Cooperative Matrix Layout`

with row-major vs column-major (and a stride operand whose meaning depends on layout), and requires memory layout operands to be constant instructions. ¹⁹

For NVIDIA's v2 path, the Vulkan repage explicitly states that the extension supports enumerating matrix types/dimensions and which features are supported, reinforcing the “query first” philosophy. ¹⁰

Synchronization and subgroup semantics still demand lockstep participation.

Baseline KHR cooperative matrices require dynamic uniformity of operands within scope, and require “all invocations active or none,” which effectively prohibits per-lane divergence around cooperative-matrix execution without careful control flow structure. ¹⁹

In Vulkan, “scope” must match queried properties, and if the scope is workgroup, the number of invocations in the local workgroup must match the reported properties—tightening the coupling between shader launch geometry and cooperative matrix semantics. ²⁶

Hardware mapping intent: use specialized matrix engines (Tensor Cores / MFMA / WMMA).

The Vulkan ecosystem positions cooperative matrices as a way to “efficiently perform matrix multiplies” by letting implementations map to hardware. ⁷

On NVIDIA, PTX ISA release notes explicitly reference `mma` shapes and `ldmatrix` as support for matrix operations and matrix loads used with MMA, providing the canonical ISA hooks expected from a backend mapping. ²⁷

On AMD, the CDNA2 ISA guide defines MFMA instructions as matrix fused multiply-add operations supporting mixed precision (dot-product style), and RDNA3 documentation explicitly presents WMMA usage via HIP compiler intrinsics—suggesting the expected AMDGPU lowering targets for cooperative matrix multiply workloads. ²⁸

Cooperative Matrix 2 vs SPV_INTEL_joint_matrix

The table below compares (a) the Cooperative Matrix 2 direction as represented by `VK_NV_cooperative_matrix2` + `SPV_NV_tensor_addressing` + SPIR-V unified opcode/capability hooks referencing `SPV_NV_cooperative_matrix2`, versus (b) `SPV_INTEL_joint_matrix` as specified in Intel's published draft spec.

Dimension	Cooperative Matrix 2 (NV path)	SPV_INTEL_joint_matrix
Primary “entry point” surface	Vulkan extension <code>VK_NV_cooperative_matrix2</code> referencing <code>SPV_NV_tensor_addressing</code> and <code>SPV_NV_cooperative_matrix2</code> , plus GLSL extension	SPIR-V extension spec <code>SPV_INTEL_joint_matrix</code> (preview) defining type + ops directly
Type model	Cooperative matrix types (baseline KHR/NV cooperative matrices) plus v2 add-ons, with additional tensor layout/view types for addressing	<code>OpTypeJointMatrixINTEL</code> defines component type, row/col, layout, and a <i>type scope</i>

Dimension	Cooperative Matrix 2 (NV path)	SPV_INTEL_joint_matrix
Memory/ addressing	Adds tensor layout/view, tensor addressing operands, and NV tensor load/store opcodes (<code>OpCooperativeMatrixLoadTensorNV</code> , <code>OpCooperativeMatrixStoreTensorNV</code>)	<code>OpJointMatrixLoadINTEL</code> / <code>Store</code> use pointer + stride + layout + <i>sync scope</i> operand
Layout repertoire	Baseline includes row/col major; v2 adds explicit tensor layout/view machinery, plus block-load hooks (<code>DecodeFunc</code>)	Layout enum includes <code>ColumnMajor</code> , <code>RowMajor</code> , and packed layouts (<code>PackedA</code> , <code>PackedB</code>) described as suitable for VNNI instructions
Ops beyond MAD/GEMM	Unified SPIR-V tables show <code>OpCooperativeMatrixReduceNV</code> and NV capability groups for conversions + per-element ops	Explicit joint-matrix MAD variants (<code>OpJointMatrixMADINTEL</code> , signed/unsigned variants) plus work-item length query
Synchronization model	Baseline cooperative matrices require dynamic uniformity within scope and “all invocations active”; Vulkan adds scope/workgroup-invocation constraints	Explicit “Scope is synchronization scope” operands on load/store (distinct from type scope)
Subgroup sizing & portability	Tightly coupled to Vulkan property queries and SPIR-V env rules; broadly aligned with Vulkan subgroup model (but v2 is NV-specific)	Tied to Intel ecosystem; portable in the sense of SPIR-V extension use, but semantics and support depend on Intel toolchains and backends
Compiler ecosystem signals	NV path is referenced by Vulkan + SPIR-V env; open compiler support still evolving	MLIR work references the Intel joint-matrix spec; LLVM SPIR-V backend docs list support for <code>SPV_INTEL_joint_matrix</code>

Sources: `VK_NV_cooperative_matrix2` repage and `SPV_NV_tensor_addressing` spec for the NV path; `SPV_INTEL_joint_matrix` spec text for Intel's model. ²⁹

Interpretation (expected performance trade-offs).

Both designs aim to reach matrix engines, but they “expose control” differently. The NV v2 direction appears oriented toward *richer address/layout control and post-/pre-processing on matrix tiles* (tensor layouts, per-element ops, reductions), which are common needs in production ML kernels. ³⁰

`SPV_INTEL_joint_matrix` emphasizes explicit layout modes (including packed A/B for VNNI) and explicit synchronization scope operands, which can make some correctness/performance intents more direct at the SPIR-V instruction level, but with portability largely gated by specific compiler/runtime stacks.

³¹

Performance limitations and top-performance strategies

Why SPV_KHR_cooperative_matrix can underperform

The abstraction is intentionally under-specified, forcing a “query + specialize” workflow.

`SPV_KHR_cooperative_matrix` does not define which `{M,N,K}` and type combinations are valid; Vulkan requires these match reported `VkCooperativeMatrixPropertiesKHR` entries (or flexible properties) or the module is invalid. This means peak performance requires choosing shapes/types that correspond to hardware-native MMA tiles, and falling back otherwise. ³²

Uniformity and full participation constraints can amplify control-flow costs.

The baseline SPIR-V extension requires cooperative-matrix operands to be dynamically uniform within the scope instance and requires that dynamic execution ensures all invocations in the scope instance are active (or none). This makes it difficult to predicatedly “mask off” lanes for edge tiles; instead, edge handling usually needs structured control flow, padding, or masked loads at the scalar level around the cooperative-matrix region. ¹

Opaque per-invocation storage can create register pressure and reduce occupancy (inference grounded in ISA realities).

Cooperative matrices are composites with an implementation-dependent per-invocation component count; implementations typically map these fragments into registers. Meanwhile, vendor tools explicitly discuss matrix instruction register usage (e.g., AMD’s matrix tooling references register usage exploration for MFMA). The practical result is that larger per-thread fragments and aggressive unrolling can significantly increase register allocation, which commonly reduces concurrent waves/warps per compute unit and can expose latency. ³³

Memory layout/stride mismatches trigger extra packing/unpacking.

For baseline cooperative matrix loads/stores, the memory layout mode and stride drive how the matrix is interpreted from linear memory, and the extension specifies stride in elements of the pointee type (which can differ from the matrix component type). Misaligned or non-native strides/layouts often force the backend to emit extra instructions to swizzle or stage data to reach a hardware instruction’s required operand packing. ³⁴

API/ISA mapping gaps are real for “non-GEMM” work.

Public cooperative-matrix proposals explicitly call out that GEMM alone is not enough for real workloads; extra manipulation around the matrix multiply is required, and baseline cooperative matrices do not directly expose those operations on opaque matrix tiles. This is a prime motivator for Cooperative Matrix 2-style extensions (conversions, reductions, per-element ops, richer addressing). ³⁵

Concrete strategies to get top performance

Strategy: treat cooperative matrix as a “microkernel primitive,” and build a classic blocked GEMM around it.

Because Vulkan exposes the supported `(M,N,K,type,scope)` combinations and ties shader validity to those, the portable high-performance path is: query supported cooperative-matrix properties → pick one or a small set of “best” tiles → specialize your kernels (or pick among precompiled variants) to match those tiles. ³⁶

Strategy: align workgroup/subgroup geometry with cooperative matrix scope.

Vulkan's SPIR-V environment rules require the scope of cooperative-matrix operands to match reported properties; when using workgroup scope, the local workgroup invocation count must match the reported `workgroupInvocations`. This means performance tuning is inseparable from launch configuration. ²⁶

Strategy: separate concerns into three layers (pack → compute → unpack), and keep the cooperative-matrix region divergence-free.

A robust pattern is: 1. **Pack** edge tiles / apply padding / reorder into a dense, hardware-friendly layout (often into shared/workgroup memory or into a contiguous scratch in device memory). 2. Run a **divergence-free cooperative-matrix loop nest** that repeatedly loads A/B tiles and issues `MulAdd` operations. 3. **Unpack / epilogue**: scale, bias, activation, reduction, etc., possibly scalar or subgroup-based if not expressible through baseline matrix ops.

This layering aligns with the explicit “opaque tile” constraints discussed in cooperative-matrix proposals. ³⁷

Strategy: schedule for memory bandwidth and avoid pathological strides.

Baseline cooperative-matrix load/store semantics emphasize layout + stride. Keeping strides unit-stride or close, and ensuring your global memory is already in a cooperative-matrix-friendly layout (or cheaply packable), reduces overhead and keeps the matrix engine fed. ¹

Strategy: have explicit fallback paths.

Because supported cooperative-matrix shapes/types are device-specific, you need fallback implementations: scalar SIMT/SIMD GEMM, subgroup shuffles + dot products, or vectorized paths. Vulkan's model expects variability and provides the query surface to decide at runtime. ³⁸

Optimization timeline diagram

```
timeline
  title Cooperative-matrix optimization timeline (practical workflow)
  section Correctness & capability gating
    Query supported VkCooperativeMatrixPropertiesKHR : gate legal MxNxK/type/
scope
    Select kernel family + specialization constants : fix tile sizes/strides/
layout
    section Baseline performance
      Implement blocked GEMM around OpCooperativeMatrixMulAddKHR : keep matrix
region divergence-free
      Coalesce loads / choose row-major vs col-major : match backend-native
layouts
    section Peak tuning
      Reduce register pressure : tune unroll, fragment reuse, split-K if needed
      Overlap memory and compute : stage tiles, pipeline loads
      Epilogue fusion strategy : keep matrix region clean, fuse outside where
possible
    section Robustness
```

Edge-tile handling & fallbacks : pad/pack or scalar fallback
Multi-device portability : build a small variant set, select by query

(Workflow constraints are driven by Vulkan’s query-and-validate model and SPIR-V cooperative-matrix semantics.) ³⁶

Kernel tiling pseudocode (“microkernel” structure)

```
// Pseudocode: high-level blocked GEMM around cooperative matrices.
// Key idea: keep the cooperative-matrix operations in a uniform, lockstep
// region.

for (tileM = 0; tileM < M; tileM += BM) {
    for (tileN = 0; tileN < N; tileN += BN) {

        // acc is a cooperative matrix tile (distributed across the chosen scope)
        coopmat_acc acc = coopmat_fill(0);

        for (tileK = 0; tileK < K; tileK += BK) {
            // Load tiles; layout/stride chosen to match the supported property
            // combination
            coopmat_a A = coopMatLoad(A_ptr + addrA(tileM, tileK), strideA, layoutA);
            coopmat_b B = coopMatLoad(B_ptr + addrB(tileK, tileN), strideB, layoutB);

            // D = A*B + C
            acc = coopMatMulAdd(A, B, acc);
        }

        // Store result tile
        coopMatStore(C_ptr + addrC(tileM, tileN), acc, strideC, layoutC);
    }
}
```

(“Coopmat” semantics, layouts, and uniformity constraints come from `SPV_KHR_cooperative_matrix`; legal shapes/types are constrained by Vulkan queries.) ³⁹

Performance tuning checklist

- Confirm tile legality by querying `VkCooperativeMatrixPropertiesKHR` (and, where relevant, flexible-dimension properties) and matching `scope` / `workgroupInvocations`. ⁴⁰
- Choose memory layouts/strides that minimize pack/unpack (row/col major choices and strides are fundamental to cooperative-matrix load/store). ¹⁹
- Keep cooperative-matrix regions structurally uniform (avoid per-lane divergence; handle edges via padding/packing or post-processing). ⁴¹

- Track register usage and occupancy effects when unrolling or increasing fragment reuse (matrix instruction tooling explicitly highlights register-usage analysis as a concern for MFMA-class operations). ⁴²
- If using Cooperative Matrix 2 features (NV path), treat tensor layouts/views as first-class: validate indexing width requirements and prefer 64-bit indexing if your tensor-address calculations would overflow 32-bit range. ⁴³

Lowering and backend mapping to NVPTX and AMDGPU

Lowering pipeline flowchart

```

flowchart TD
    A[High-level shader / kernel IR] --> B[SPIR-V with cooperative matrix ops]
    B --> C{Extension set?}
    C -->|SPV_KHR_cooperative_matrix| D[Canonicalize to KHR coopmat ops]
    C -->|SPV_NV_cooperative_matrix2 + SPV_NV_tensor_addressing| E[Canonicalize tensor layouts/views + NV coopmat ops]
    D --> F[Mid-level "matrix tile" IR / intrinsics]
    E --> F
    F --> G{Target backend}
    G -->|NVPTX| H[Lower to NVVM/NVPTX intrinsics -> PTX mma/lmatrix/...]
    G -->|AMDGPU| I[Lower to llvm.amdgcn MFMA/WMMA intrinsics -> AMD ISA]
    H --> J[Assemble/JIT to native GPU ISA]
    I --> J

```

(The existence of NVVM/NVPTX intrinsics and their role in lowering is reflected in MLIR's NVVM dialect documentation; AMDGPU lowering relies on AMDGPU backend intrinsics and documented MFMA/WMMA ISA features.) ⁴⁴

Baseline SPV_KHR cooperative matrix → NVPTX mapping (Ampere+)

Relevant ISA hooks.

NVIDIA's PTX ISA documentation describes PTX as the virtual ISA, and PTX ISA release notes explicitly mention the `mma` instruction (with known shapes) and `ldmatrix` instructions for loading matrices from shared memory for MMA. ⁴⁵

At the compiler-IR level, MLIR's NVVM dialect explicitly includes an `nvvm.mma.sync` op, reflecting the intrinsic-level mapping point used before PTX emission. ⁴⁶

(Conceptually: SPIR-V cooperative-matrix `MulAdd` → NVVM/NVPTX MMA intrinsic → PTX `mma.sync` or newer variants.)

Concrete lowering pattern (conceptual).

A typical high-performance lowering for `OpCooperativeMatrixMulAddKHR` on NVIDIA involves: 1. **Stage** operand tiles into shared memory in a layout appropriate for `ldmatrix` (when the operand sources are in global memory), using cooperative global loads by the warp/subgroup. 2. Use `ldmatrix` to load sub-tiles from shared into registers in the precise fragment packing the hardware expects. 3. Emit `mma.sync.*`

instructions to perform the matrix multiply-accumulate. 4. For stores, reverse the staging where needed (register → shared/global), with layout considerations.

The PTX ISA release notes explicitly connect `ldmatrix` with MMA usage, supporting the idea that correct and fast lowering depends on staging and specialized matrix loads. ²⁷

Mixed precision and saturation flags.

Baseline KHR cooperative-matrix `MulAdd` includes optional operand flags such as `SaturatingAccumulationKHR` and signedness controls for integer components, with detailed integer semantics described in the extension. Vulkan additionally exposes whether saturating accumulation must or must-not be requested for a supported combination. ⁴⁷

A correct NVPTX lowering must therefore choose the PTX MMA variant(s) compatible with the `(AType,BType,CType,ResultType)` combination and enforce the saturating/signedness rules requested/required. ⁴⁸

Baseline SPV_KHR cooperative matrix → AMDGPU mapping (RDNA3/CDNA2+)

Relevant ISA hooks.

The AMD CDNA2 ISA guide describes MFMA instructions and emphasizes that they perform dot-product style operations with mixed precision, naming conventions that encode tile shape and operand formats. ⁴⁹

For RDNA3, AMD’s WMMA guidance explicitly frames WMMA as a feature usable with HIP via compiler intrinsics, implying a lowering path where cooperative-matrix ops become AMDGPU WMMA/MFMA intrinsics depending on architecture. ⁵⁰

LLVM’s AMDGPU documentation includes intrinsic-level controls for scheduling and mentions MFMA-related scheduling group controls and MFMA-scale intrinsics, which are relevant for instruction scheduling around MFMA-heavy inner loops. ⁵¹

Concrete lowering pattern (conceptual).

A practical AMDGPU backend lowering strategy mirrors NVIDIA in structure but targets wave-based matrix ops: 1. Map each cooperative-matrix object to a set of VGPR fragments for each lane in a wave/subgroup (the exact fragment mapping is hardware/ISA specific). ⁵²

2. Emit `llvm.amdgcn.mfma.*` intrinsics (CDNA2+) or WMMA intrinsics (RDNA3), selecting shape/type variants that match the Vulkan-reported supported `(M,N,K,types)` combination. ⁵³

3. Use scheduler guidance (where available and beneficial) to keep MFMA sequences dense while interleaving memory ops to hide latency. ⁵⁴

NVPTX vs AMDGPU intrinsic mappings and codegen challenges

Aspect	NVPTX (NVIDIA)	AMDGPU (AMD)
Matrix compute instruction family	PTX <code>mma</code> is the core hook; <code>ldmatrix</code> supports operand loads for MMA	CDNA2 MFMA family; RDNA3 WMMA feature via HIP intrinsics

Aspect	NVPTX (NVIDIA)	AMDGPU (AMD)
Thread grouping	Warp-level (32 threads) is the common unit; newer variants exist for larger groups	Wavefront-level (implementation-defined wave size) is central; mapping depends on architecture
Operand packing	Highly specific fragment layouts; often benefits from shared-memory staging before matrix loads	Highly specific lane/register mapping; tooling emphasizes that mapping and register usage can be nontrivial
SPIR-V “scope” mapping	Typically subgroup scope aligns with warp; Vulkan requires scope match properties	Typically subgroup scope aligns with wave; Vulkan requires scope match properties
Major codegen pitfall	Layout/stride mismatches causing extra permutes/staging; edge tiles can’t be lane-masked inside coopmat region	Fragment mapping and VGPR pressure; tuning instruction scheduling around MFMA-heavy regions
Cooperative Matrix 2 “tensor addressing”	Likely maps to advanced addressing + block load paths; Vulkan env notes 64-bit indexing rules	Similar addressing complexity; may require explicit pack/transpose depending on native WMMA/MFMA expectations

(ISA-level and ecosystem hooks are evidenced by PTX and AMD ISA documentation, plus AMD WMMA guidance and Vulkan’s scope/property constraints.) ⁵⁵

CUDA / HIP builtins → SPIR-V cooperative-matrix instruction mapping (conceptual, not compiler-internals)

This section intentionally focuses on *semantic correspondences* rather than a specific end-to-end compiler pipeline, as requested.

CUDA WMMA / MMA-style builtins (conceptual) → SPIR-V

NVIDIA’s PTX ISA release notes show the existence of `wmma.mma` and `mma` instruction families for tensor-core matrix operations, and the `ldmatrix` instruction to load matrices from shared memory for MMA. ²⁷

Given that, the closest SPIR-V correspondences for a CUDA WMMA-style program are:

- `wmma::load_matrix_sync` / “load fragment” → `OpCooperativeMatrixLoadKHR` (baseline) or `OpCooperativeMatrixLoadTensorNV` (v2 tensor addressing path) depending on whether tensor layouts/views are being used. ⁵⁶
- `wmma::mma_sync` / MMA intrinsic → `OpCooperativeMatrixMulAddKHR`. ⁵⁷
- `wmma::store_matrix_sync` → `OpCooperativeMatrixStoreKHR` (or tensor store NV variant if using tensor addressing). ⁵⁸
- Epilogue operations performed per element on fragments (bias, activation, scaling) → baseline: must generally be done via allowed arithmetic/conversion instructions on cooperative matrix

composites (where legal) or by extracting/scalarizing; v2 direction: intended to be covered more directly via per-element operations and conversion capability groups linked to `SPV_NV_cooperative_matrix2`. ⁵⁹

HIP WMMA / MFMA intrinsics → SPIR-V

AMD's RDNA3 WMMA documentation explicitly describes WMMA usage in HIP via compiler intrinsics, providing a natural semantic mapping point to SPIR-V cooperative matrix operations. ⁵⁰

Conceptually: - HIP WMMA "load fragment" intrinsic → `OpCooperativeMatrixLoadKHR` / `LoadTensorNV` (depending on addressing model). ⁶⁰

- HIP WMMA "mma" intrinsic or MFMA intrinsic sequence → `OpCooperativeMatrixMulAddKHR`. ⁶¹

- HIP WMMA "store fragment" intrinsic → `OpCooperativeMatrixStoreKHR` / `StoreTensorNV`. ⁵⁸

Practical caveat: "builtins → SPIR-V" is not a standard direct compilation path today

Even though the semantic mapping is straightforward, open-source toolchain support for emitting `SPV_NV_cooperative_matrix2` and `SPV_NV_tensor_addressing` from LLVM-based pipelines is still an active topic—e.g., an open `SPIRV-LLVM-Translator` issue asks whether these extensions are in scope, indicating that production-ready direct emission/translation may not yet be ubiquitous in that ecosystem. ⁵

¹ ⁶ ¹³ ¹⁹ ³² ³³ ³⁴ ³⁹ ⁴¹ ⁴⁷ ⁴⁸ ⁵⁶ ⁵⁷ ⁵⁸ ⁵⁹ ⁶⁰ ⁶¹ https://github.com/KhronosGroup/SPIRV-Registry/blob/main/extensions/KHR/SPV_KHR_cooperative_matrix.asciidoc

https://github.com/KhronosGroup/SPIRV-Registry/blob/main/extensions/KHR/SPV_KHR_cooperative_matrix.asciidoc

² ⁸ ³⁶ ⁴⁰ <https://docs.vulkan.org/refpages/latest/refpages/source/VkCooperativeMatrixPropertiesKHR.html>

<https://docs.vulkan.org/refpages/latest/refpages/source/VkCooperativeMatrixPropertiesKHR.html>

³ ¹⁰ ¹⁸ ²⁹ https://docs.vulkan.org/refpages/latest/refpages/source/VK_NV_cooperative_matrix2.html

https://docs.vulkan.org/refpages/latest/refpages/source/VK_NV_cooperative_matrix2.html

⁴ ²⁷ ⁵⁵ https://docs.nvidia.com/cuda/pdf/ptx_isa_8.5.pdf

https://docs.nvidia.com/cuda/pdf/ptx_isa_8.5.pdf

⁵ <https://github.com/KhronosGroup/SPIRV-LLVM-Translator/issues/3472>

<https://github.com/KhronosGroup/SPIRV-LLVM-Translator/issues/3472>

⁷ https://docs.vulkan.org/refpages/latest/refpages/source/VK_KHR_cooperative_matrix.html

https://docs.vulkan.org/refpages/latest/refpages/source/VK_KHR_cooperative_matrix.html

⁹ ²⁶ <https://vulkan.lunarg.com/doc/view/1.4.304.1/mac/antora/spec/latest/appendices/spirvenv.html>

<https://vulkan.lunarg.com/doc/view/1.4.304.1/mac/antora/spec/latest/appendices/spirvenv.html>

¹¹ ²¹ ²³ <https://registry.khronos.org/SPIR-V/specs/unified1/SPIRV.pdf>

<https://registry.khronos.org/SPIR-V/specs/unified1/SPIRV.pdf>

¹² ³⁰ ⁴³ https://github.com/khronos.org/SPIRV-Registry/extensions/NV/SPV_NV_tensor_addressing.html

https://github.com/khronos.org/SPIRV-Registry/extensions/NV/SPV_NV_tensor_addressing.html

14 31 https://github.com/intel/llvm/blob/c2f8602c843d2ba87dc1c9057f1422f23a77ed66/sycl/doc/design/spirv-extensions/SPV_INTEL_joint_matrix.asciidoc

https://github.com/intel/llvm/blob/c2f8602c843d2ba87dc1c9057f1422f23a77ed66/sycl/doc/design/spirv-extensions/SPV_INTEL_joint_matrix.asciidoc

15 <https://reviews.llvm.org/D131586>

<https://reviews.llvm.org/D131586>

16 <https://llvm.org/docs/SPIRVUsage.html>

<https://llvm.org/docs/SPIRVUsage.html>

17 <https://github.com/microsoft/vscode-cpptools/blob/main/Extension/CHANGELOG.md>

<https://github.com/microsoft/vscode-cpptools/blob/main/Extension/CHANGELOG.md>

20 35 37 https://github.com/khronos.org/Vulkan-Site/features/latest/features/proposals/VK_QCOM_cooperative_matrix_conversion.html

https://github.com/khronos.org/Vulkan-Site/features/latest/features/proposals/VK_QCOM_cooperative_matrix_conversion.html

22 <https://registry.khronos.org/SPIR-V/specs/unified1/SPIRV.html>

<https://registry.khronos.org/SPIR-V/specs/unified1/SPIRV.html>

24 <https://docs.vulkan.org/spec/latest/appendices/spirvenv.html>

<https://docs.vulkan.org/spec/latest/appendices/spirvenv.html>

25 38 <https://docs.vulkan.org/spec/latest/chapters/shaders.html>

<https://docs.vulkan.org/spec/latest/chapters/shaders.html>

28 49 52 53 <https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/instruction-set-architectures/instinct-mi200-cdna2-instruction-set-architecture.pdf>

<https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/instruction-set-architectures/instinct-mi200-cdna2-instruction-set-architecture.pdf>

42 <https://gpuopen.com/learn/amd-lab-notes/amd-lab-notes-matrix-cores-readme/>

<https://gpuopen.com/learn/amd-lab-notes/amd-lab-notes-matrix-cores-readme/>

44 46 <https://mlir.llvm.org/docs/Dialects/NVVM/Dialect/>

<https://mlir.llvm.org/docs/Dialects/NVVM/Dialect/>

45 <https://docs.nvidia.com/cuda/parallel-thread-execution/>

<https://docs.nvidia.com/cuda/parallel-thread-execution/>

50 https://gpuopen.com/learn/wmma_on_rdna3/

https://gpuopen.com/learn/wmma_on_rdna3/

51 54 https://llvm.org/docs/_sources/AMDGPUUsage.rst.txt

https://llvm.org/docs/_sources/AMDGPUUsage.rst.txt